Inventory Architecture Review

Purpose and scope

This review compares the pump, motor, and compressed-air inventory features as implementation references for a future Fan Inventory. It describes the current Angular application patterns, notes where the three features differ, and identifies practices and risks relevant to extending the application. It is an architecture review, not a specification of fan fields or calculation behavior.

The reviewed implementations are under:

- `pump-inventory`
- `motor-inventory`
- `compressed-air-inventory`
- Shared inventory persistence and dashboard code under `shared/models/inventory` and `dashboard`

The application also has fan calculators under `calculator/fans` and a Fan System Assessment (FSAT) under `fsat`. These are related fan-domain features, but neither is the same thing as a persistent, directory-managed fan inventory.

Architecture at a glance

Shared application shell and persistence

All three inventories are represented as `InventoryItem` records in the common IndexedDB inventory store. The shared model currently carries a `type` discriminator and one optional, feature-specific data property for each inventory (`motorInventoryData`, `pumpInventoryData`, or `compressedAirInventoryData`); inventory settings are stored separately and associated by inventory ID. `InventoryDbService` provides the shared add, update, delete, and lookup operations.

Creation is coordinated by the dashboard. `CreateInventoryComponent` selects a feature-specific factory on `InventoryService`, which supplies the initial feature data, and then creates associated settings. The directory dashboard and sidebar select a feature route from the inventory type/data. Editing, duplication, moving, and deletion also live in shared dashboard code. Adding an inventory is consequently a cross-cutting change, not just a new feature module.

Feature boundaries

Each inventory has a feature directory containing a typed domain model, inventory service, entry component, routing, and Angular modules. Setup and summary views are composed from feature-specific child components and services. The modules reuse shared facilities such as settings, forms, help panels, import/export, connected inventory UI, and exportable results tables.

The entry component loads the inventory and its settings, keeps the feature state synchronized, and writes changes through `InventoryDbService`. The feature service provides view state and domain operations. The separation is useful, but the component also coordinates persistence, route state, and connected-inventory behavior; a new feature should avoid growing this component into an all-purpose domain service.

Model and user-flow patterns

Pump and motor: department/catalog hierarchy

Pump and motor data use a common shape: inventory data contains departments, and each department has a catalog of equipment items. Items carry IDs and a parent department ID, and models group fields into domain-specific subobjects. For example, pump items divide fluid, nameplate, equipment, motor, status, system, and field-measurement data. Motor items group nameplate, operating, manual-specification, load-characteristic, purchase, torque, and batch-analysis data.

Both models include display-option objects, allowing users to select which property groups or fields appear in the summary. Their services initialize a default department and starter item, create default display options, and filter the summaries by equipment properties. Pump also records per-item and per-department validation state.

Compressed air: system/catalog hierarchy plus end uses

Compressed-air inventory organizes compressor catalog items under systems rather than departments. Systems also hold system-level properties and end uses. Compressor data is partitioned into nameplate, motor, controls, design, performance-points, centrifugal-specifics, and field-measurement groups. Display options and summary filters follow the same broad idea as pump and motor, but are tailored to compressors and systems.

This is a useful precedent when fan inventory requirements are system-oriented or when a fan is associated with multiple operating areas. The correct hierarchy should come from fan-inventory use cases, rather than copying one of these models without review.

Routes, tabs, and summaries

Pump and compressed-air inventory use route-derived state for setup and summary views. Their route definitions include named child paths, and services maintain the current view in `BehaviorSubject`s. Compressed air defines ordered view constants and has explicit route gating for a valid inventory; pump has a related, simpler tab model. Motor currently has a distinct route shape: setup and summary are parent routes, with a separate analysis route.

All three provide summary experiences, but their contents and service organization are feature-specific. Pump and compressed-air summary modules expose overview, graphs, and table views; motor has similar summary views and also has batch analysis. Shared exportable table components are reused where appropriate.

Validation and unit conversion

Inventory services and catalog/form services own domain validity checks. Pump creates forms from the motor, field-measurement, and equipment groups and stores item/department/inventory validity. Compressed air builds a validity result across compressor data groups and system information. Validity is used by route/banner flows to determine when the user may proceed.

Each feature has its own converter service for translating stored numeric values when settings change between Metric and Imperial. Conversion is field-specific and uses shared `ConvertUnitsService`; the conversion code also rounds values. This makes model completeness and explicit unit mapping essential. New fan data must distinguish display-only choices from actual persisted-value conversions and must preserve the application's unit conventions.

Shared application integration points

The following existing files and responsibilities should be reviewed when adding Fan Inventory:

Concern	Existing entry points
Persisted union-like inventory shape	`shared/models/inventory/inventory.ts`, `indexedDb/inventory-db.service.ts`
New inventory and starter feature data	`dashboard/inventory.service.ts`, `dashboard/create-inventory/create-inventory.component.ts`
Application route and feature module registration	`core/core.routing.ts`, plus the core module's imports
Directory cards and routing	`dashboard/directory-dashboard/directory-items/inventory-item`, `dashboard/sidebar/inventory-item`
Copy, move, and delete behavior	`dashboard/copy-items/copy-items.component.ts`, directory inventory item component
Inventory file import/export	`shared/import-export/export.service.ts`, `shared/import-export/import.service.ts`
User settings and unit preferences	`indexedDb/settings-db.service.ts`, feature converter service
Cross-feature links	`shared/connected-inventory` and the related assessment integration flows

The generic inventory import/export flow currently transports the inventory object and its settings together. It clears storage IDs and assigns new IDs on import, while the copy path deep-copies records and has explicit feature-specific cleanup for existing links. The generic transport reduces the need for inventory-specific serialization, but it does not automatically guarantee that nested IDs, links, or older/malformed data are handled correctly.

Practices to carry forward

- Keep a strongly typed feature model and isolate feature behavior in its own service/module rather than placing domain logic in dashboard components.
- Use the shared inventory record, IndexedDB service, settings model, directory UI, and import/export pathway instead of creating parallel persistence or navigation systems.
- Partition complex item data into meaningful groups and make required, optional, and conditional fields explicit.
- Define the supported hierarchy and IDs deliberately; maintain stable parent-child relationships when adding, deleting, filtering, copying, or importing items.
- Centralize form construction and validation so the setup flow, route gating, and summaries agree on validity.
- Provide an explicit initial data factory and display defaults. Keep the default item usable and ensure an empty or partially completed inventory has a deliberate experience.
- Give every persisted numeric field an explicit unit decision and cover both directions of settings conversion, including absent values.
- Reuse common presentation controls where the interaction is genuinely shared, while keeping fan-specific fields, summaries, and validation in the fan feature.
- Add focused specs for route structure and important user-visible behavior. Existing pump and compressed-air route specs illustrate testing route tab structure.

Differences and risks to account for

- **The patterns are not a single consistent template.** Pump and compressed air have route-derived nested views; motor's routing and analysis flow differ. Prefer the closest domain fit, but define a consistent fan route contract and test it.
- **Feature state is commonly mutated in place.** Services use `BehaviorSubject`s and often mutate nested objects before emitting them. A new feature should follow existing observable ownership while avoiding stale references and accidental edits to persisted/copy source data.
- **Persistence types form an expanding union by convention.** Adding a fan data property and type string requires updating consumers that branch on the current three types; overlooking a dashboard or utility branch can make otherwise-valid inventory data inaccessible.
- **Copy and connection cleanup are explicit.** Copying an inventory with linked assessment/inventory references may create invalid cross-record links. Decide which connections are removed, recreated, or retained and implement that intentionally.
- **Unit conversion is schema-sensitive.** A missing field mapping can silently leave stored values in the old unit system. Optional/nested values and unit settings need explicit round-trip tests.
- **Persisted shapes outlive the current UI.** IndexedDB data and exported backup files can be older than the deployed application. Keep new feature data version-tolerant and decide whether normalization or migration is needed before releasing schema changes.

- **A fan inventory has adjacent features but no current inventory precedent.** FSAT and fan calculators provide domain context, while shared connected-inventory contracts currently identify motor, pump, and compressed-air inventory types. Reuse or extension of these contracts—and any FSAT integration—requires an explicit design decision, not an assumed existing fan link.
- **Feature breadth can become costly.** The mature inventories include setup, validation, filters, overview, graphs, tables, conversion, help, export, and integration flows. Set an MVP scope for Fan Inventory and defer optional capabilities deliberately rather than leaving partially wired paths.

Review conclusion

Fan Inventory should be implemented as a first-class inventory feature within the shared dashboard, settings, and IndexedDB architecture, with fan-specific data types and presentation behavior. The department/catalog and system/catalog patterns are alternatives for domain modeling, not mandates. The safest foundation is to decide the fan inventory's hierarchy, intended relationship to FSAT, data/unit contract, and MVP summary needs before extending the cross-cutting inventory type branches. The companion implementation plan turns these observations into sequenced work.